Concept Question Answers

Category B Practice - Part C, Questions 1-10

1. Overloading and overriding

Overloading uses the same method name with different parameter lists; overload selection occurs at compile time. Overriding supplies a subtype implementation with the same signature; dynamic dispatch selects it at run time based on the actual object.

2. Constructor invocation

When a derived object is created, base constructors run before derived constructors. In Java, pass values to the base constructor with super(arguments), as the first constructor statement.

3. Concrete classes, abstract classes, interfaces

Concrete classes can be instantiated and hold state and implementation. Abstract classes cannot be instantiated directly and can combine fields, constructors, concrete methods and abstract methods. Interfaces define contracts/capabilities, making them appropriate for shared behavior across unrelated types.

4. Identical methods in two interfaces

If two interfaces declare the same compatible abstract method signature, one public implementation in the class satisfies both. This demonstrates that a class may combine multiple interface contracts.

5. equals() and hashCode()

Objects considered equal by equals() must return the same hashCode(). Otherwise HashSet and HashMap may fail to find equal keys or may allow logical duplicates because equal objects can map to different buckets.

6. Shallow and deep copying

A shallow copy copies references to nested mutable objects, so two copies may share and mutate the same nested object (such as an Address). A deep copy duplicates those nested mutable objects so each copy can change independently.

7. Inner and static nested types

A non-static inner class is tied to an enclosing instance and can access its instance members. A static nested class needs no enclosing instance and directly accesses only static members of the outer class. An instance-bound iterator is a suitable inner class; a node/helper type can be static nested.

8. Array operation complexity

Index access is O(1), because the element address is computed directly from the base address and index. Middle insertion/deletion is O(n) because later elements must usually be shifted to open or close a gap.

9. Polymorphic arrays and dynamic dispatch

An array of a base type can store different derived objects. Calling an overridden method through the base reference runs the implementation belonging to each actual object, so explicit type checks are unnecessary.

10. IS-A or CAN-DO

Use inheritance for a genuine subtype relationship, such as Car IS-A Vehicle. Use interfaces to express a capability shared by unrelated types, such as Car, Phone, and Watch implementing Chargeable.